

Variables and Types

CSC 230 : C and Software Tools

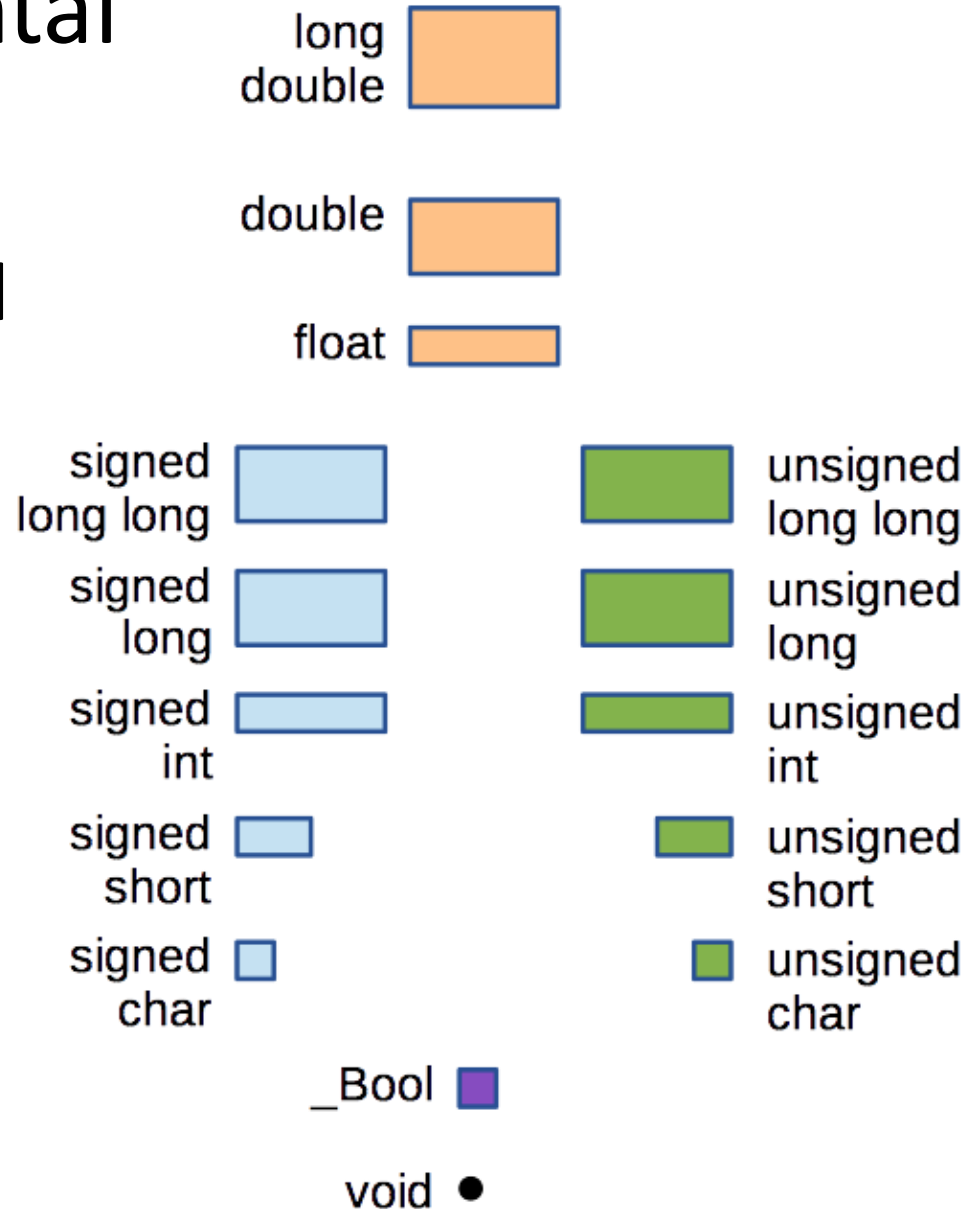
NC State Department of Computer
Science

Topics for Today

- Built-in Types
- Variables and Scope
- Boolean
- Operators and Precedence
- Usual arithmetic conversion

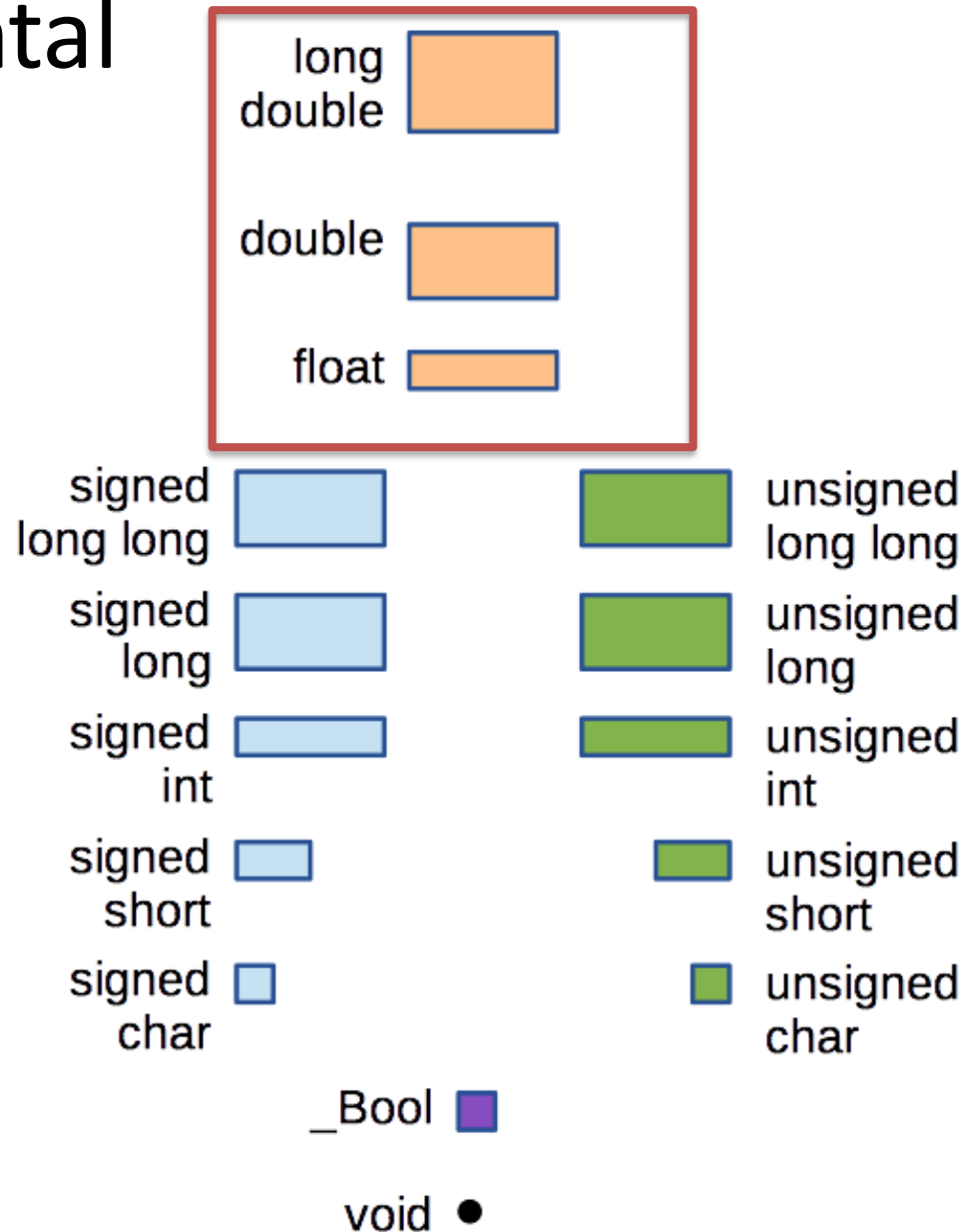
C Fundamental Types

- This figure shows all the basic types we have.
- ... a few more than we have in Java.



C Fundamental Types

- You have types for real (floating point) values.
 - Including **long double**, which we don't have in Java



Integer Types

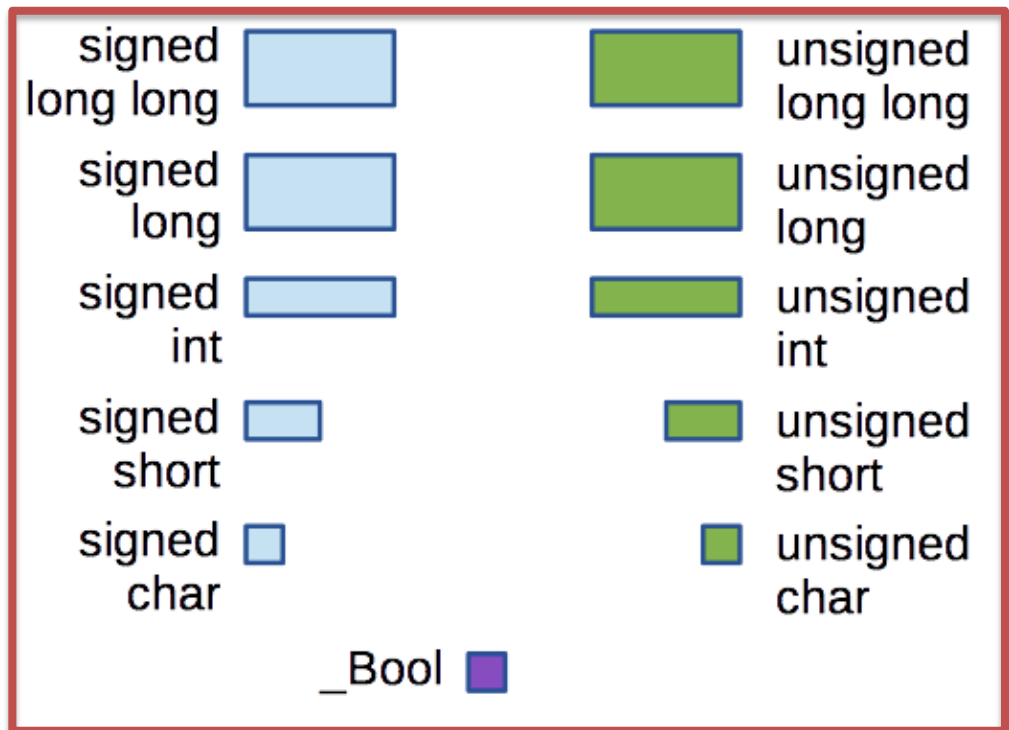
- Most of these are for storing integers.
- There many integer type *specializations* for:
- Different sizes
 - Including long long, which we don't have in Java
- Different signed-ness
 - Mostly, signed is the default.
 - Except for char

Signed-ness of char is platform dependent.

long
double 

double 

float 



void •

What's in a Name

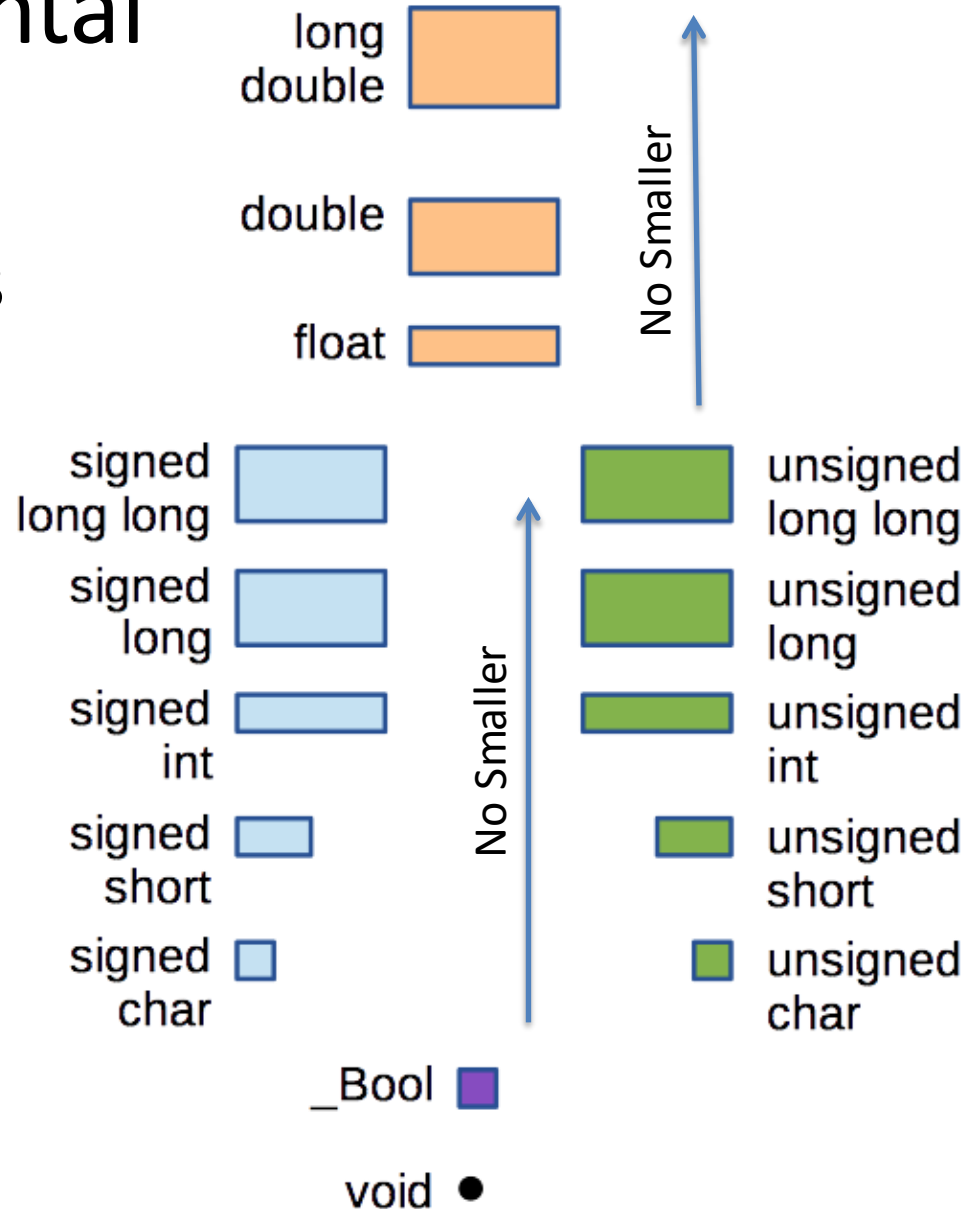
- There are different ways to name the same integer type.
- Signed is usually the default, so
 - `int` is the same as `signed int`
 - `long` is the same as `signed long`
 - `short` is the same as `signed short`
 - ...

What's in a Name

- The word int is optional for short, long and long long, so:
 - long int is the same as long
 - long long int is the same as long long
 - short int is the same as short
 - ...
- So, there are multiple names for the same type:
 - signed long int is the same as long
 - unsigned short int is the same as unsigned short
 - ...

All Fundamental Types

- The standard makes few guarantees about the sizes of these types.
 - For example, long may or may not have more capacity than int
 - ..., but it must not have less capacity.



Integer Capacity

- The exact range of a type is platform dependent
- In general, we can figure this out based on the memory used for each variable type.
- Or, we can just look in the `limits.h` header
 - It's a file: `/usr/include/limits.h` (on the common platform)
- With constant names:
 - `SHRT_MIN`, `SHRT_MAX`, `USHRT_MAX`
 - `INT_MIN`, `INT_MAX`, `UINT_MAX`
 - `LONG_MIN`, `LONG_MAX`, `ULONG_MAX`



Signed range



Unsigned range

Integer Capacity

On the Common Platform

Type	Bits	Minimum	Maximum
signed char	8	-128	127
unsigned char	8	0	255
signed short	16	-32,768	32,767
unsigned short	16	0	65,535
signed int	32	-2,147,483,648	2,147,483,647
unsigned int	32	0	4,294,967,295
signed long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long	64	0	18,446,744,073,709,551,615
signed long long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long long	64	0	18,446,744,073,709,551,615

Integer Capacity

On the Common Platform

Type	Bits	Minimum	Maximum
signed char	8	-128	127
unsigned char	8	0	255
signed short	16	-32,768	32,767
unsigned short	16	0	65,535
signed int	32	-2,147,483,648	2,147,483,647
unsigned int	32	0	4,294,967,295
signed long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long	64	0	18,446,744,073,709,551,615
signed long long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long long	64	0	18,446,744,073,709,551,615

See, same capacity as long (but
no smaller than long)

Real Number Capacity On the Common Platform

Type	Bits	Precision	Range
float	32	23 bits (about 7 digits)	About 10^{-38} .. 10^{38}
double	64	52 bits (about 15 digits)	About 10^{-308} .. 10^{308}
long double	128	64 bits (about 19 digits)	About 10^{-4930} .. 10^{4930}

Real Number Capacity On the Common Platform

Actually less memory than long
or long long.

Same as Java

Type	Bits	Precision	Range
float	32	23 bits (about 7 digits)	About $10^{-38} .. 10^{38}$
double	64	52 bits (about 15 digits)	About $10^{-308} .. 10^{308}$
long double	128	64 bits (about 19 digits)	About $10^{-4930} .. 10^{4930}$

An extra type Java doesn't
have.

Real Number Capacity On the Common Platform

Way smaller than an
electron (in meters)

Way larger than the observable
universe (in meters)

Type	Bits	Precision	Range
float	32	23 bits (about 7 digits)	About 10^{-38} .. 10^{38}
double	64	52 bits (about 15 digits)	About 10^{-308} .. 10^{308}
long double	128	64 bits (about 19 digits)	About 10^{-4930} .. 10^{4930}

This is where larger floating-
point types spend their bits.

Making Variables

- *A variable* has:
 - A **name** : that's a legal identifier name
 - A **type** : that's how the compiler interprets the contents of memory representing the variable
 - A **scope** : that's the section of code that can access the variable
 - A **storage class** : that's how it's stored and initialized

Static vs Dynamic Types

- In C (like Java), all variables are *statically typed*
 - The type of a variable type can't change as the program runs
- It wouldn't have to be this way
 - Lots of languages have *dynamic typing* (e.g., JavaScript, PHP, Python)
- It's a tradeoff in:
 - flexibility
 - performance
 - ability to detect errors at compile time

Auto (Local) Variable Scope

- Declared inside a function
 - ... including function parameters.
- In scope from the declaration to the end of the surrounding block
- These are called *auto* variables
 - Their storage class is called *auto*
 - In fact, auto is a keyword in C

```
void snap( int n )
{
    auto int i = 0;

    while ( i < n ) {
        char j = 'a' + i;
        ...;
        i++;
    }
    if ( i > 20 ) {
        float j = 0.0;
        ...;
    }
}

int crackle( int x, int y )
{
    int j = x * y;
    ...;
}
```

Global Variable Scope

- A variable defined outside any function has *global scope*
- Its lifetime is the whole execution of the program
 - Their storage class is called *static*
- Any code can access this variable.
- There's a potential for name collision

```
int x = 25;

int main(void)
{
    ... x ...
}

int length = 15;

int f(...)
{
    ... x ... length ...
}
```

Shadowing

- Two variables can have the **same name** ... as long as they are declared in **different scopes**.
 - They are independent variables, just with the same name.
- How does the compiler decide which one you want?
 - You get the one in the one declared in the narrowest surrounding scope.
- We say the one in the narrower scope **shadows** the one in the wider scope

Shadowing Example

```
int x = 6;
int y = 8;

void waffle( int z )
{
    for ( int x = z; x < y; x++ ) {
        int z = x + y;
        if ( z <= 10 ) {
            int x = z * 2;
            printf( "%d %d %d\n", x, y, z );
        }
    }

    int y = x - 1;
    printf( "%d %d %d\n", x, y, z );
}
```

Shadowing Example

```
int x = 6;
int y = 8;

void waffle( int z )
{
    for ( int x = z; x < y; x++ ) {
        int z = x + y;
        if ( z <= 10 ) {
            int x = z * 2;
            printf( "%d %d %d\n", x, y, z );
        }
    }

    int y = x - 1;
    printf( "%d %d %d\n", x, y, z );
}
```

The diagram illustrates variable shadowing with red arrows. An arrow points from the global variable `y` (value 8) to the `y` parameter in the `printf` call inside the `waffle` function. Another arrow points from the global variable `x` (value 6) to the `x` parameter in the `printf` call inside the `waffle` function. A third arrow points from the `int z` declaration inside the `for` loop to the `z` parameter in the `printf` call inside the `if` block. A fourth arrow points from the `int x` declaration inside the `if` block to the `x` parameter in the `printf` call inside the `if` block. These arrows show how local declarations shadow global ones within their respective scopes.

Shadowing Example

```
int x = 6;
int y = 8;

void waffle( int z )
{
    for ( int x = z; x < y; x++ ) {
        int z = x + y;
        if ( z <= 10 ) {
            int x = z * 2;
            printf( "%d %d %d\n", x, y, z );
        }
    }

    int y = x - 1;
    printf( "%d %d %d\n", x, y, z );
}
```

The diagram illustrates variable shadowing with red arrows. An arrow points from the global variable `x` (value 6) to the `x` parameter of the `waffle` function. Another arrow points from the global variable `y` (value 8) to the `y` argument in the `printf` call at the end of the function. Inside the `for` loop, an arrow points from the local `x` (value `z`) to the `x` argument in the `printf` call. Inside the `if` block, an arrow points from the local `z` (value `x + y`) to the `z` argument in the `printf` call. This shows how local variables can shadow global ones, and how the same variable name can refer to different memory locations within the same scope.

Shadowing

- Scoping rules let you shadow variables in lots of languages
 - e.g., in Java, a local variable can shadow a field.
- This is a clever trick ... that you should probably avoid using too cleverly.
- There's at least one place where it's nice to have.
 - In macro definitions ... later.

Literal Values

- For variables, the declaration determines the type.

```
int x = 45;  
double y = 17;
```

- For numbers, the compiler will decide on a type

```
f( 315, -1.45 )
```

What type will I
have?

How about me?

Integer Literal Values

- For a sequence of digits, the compiler assumes you want the int type
- Unless the value is too large to fit in an int
 - ... then, it assumes you want the smallest integer type that's big enough:
 - Signed int if the value is not too big (e.g., 75)
 - Signed long if the value is too big for int (e.g., 5271486214)
 - Signed long long if it's too big to be a long

Integer Literal Values

- You can say what type you want for literal values
 - For unsigned, put a U at the end (e.g., 351**U**)
 - For long, put an L at the end (e.g., 351**L**)
 - For long long, put LL at the end (e.g., 351**LL**)
 - You can combine these, in any order (e.g., 351**ULL**)

Floating Point Literals

- With a decimal or scientific notation, the compiler assumes **double**
 - Like this: 3.14 (or 3.0 or 0.14 or .14)
 - Or in scientific notation: 6.022E23
- You can add a character to specify the type
 - The smaller, **float** type: 3.14F
 - The larger, **long double** type: 3.14L
- Does the type on a literal matter?

The C99 Boolean Type

- Originally, C had no specific boolean type
- C would let you use any numeric type:
 - With 0 being interpreted as false
 - And anything else being true.
- This still works just fine.

```
int i = 100;  
while ( i ) {  
    i -= 1;  
}
```

```
double f = 10;  
while ( f ) {  
    f -= 0.1;  
}
```

It even works with floating-point types ... but this is probably a bad idea.

A Time Before Boolean

- Earlier, we pretended functions like `isalpha()` returned `bool`.
 - This was a lie.

 `bool isalpha( int ch );`

- These functions pre-date the `bool` type.
 - They really return `int`.

```
int isalpha( int ch );  
int isspace( int ch );  
int isdigit( int ch );  
...
```

About `_Bool`

- In C99, they added a type for boolean values.
- They had to give it a goofy name, `_Bool`, a name nobody would have been using for anything.
- Still, internally, boolean values are
 - 0 for false
 - Anything else for true
 - With 1 as the typical value for true.

```
_Bool b;  
b = 0;  
b = 1;  
b = 100;
```

I'm false.

I'm true.

I'll still just be
1.

Meet stdbool.h

- `_Bool` works, but it looks like a strange type name.
- You can include **`stdbool.h`**
 - You get the name the name **`bool`** as an alias for **`_Bool`**
 - But wait, there's more. You also get
 - **`false`** : a preprocessor constant for 0
 - **`true`** : a preprocessor constant for 1
- This header lets you code as if `bool` is a built-in type

Conditional Expressions

- The logical operators (**!**, **&&** and **||**) were defined we had a `_Bool` type.
- They expect **int** parameters
 - Zero $\rightarrow$ false
 - Anything else $\rightarrow$ true
- Evaluate to an **int** result
 - Zero $\rightarrow$ false
 - One $\rightarrow$ true

```
int i = 276, j = 0;  
float f = 2.718;
```

```
x = i || j;
```

This evaluates to an
int (with value 1)

```
y = f && j;
```

This also evaluates to
an int (with value 0)

Short Circuiting

- The `&&` and `||` operators are guaranteed to be short circuiting (just like Java)
 - This can improve performance; we stop evaluating early if the outcome is determined.
 - More importantly, it lets us guard some tests with other tests that make sure they are safe.

This only gets
evaluated ...

... if this is true.

```
if ( people != 0 && slices / people >= 2 ) {  
    printf( "Everyone can have two slices!\n" );  
}
```

C Expressions

- C and Java have many of the same operators
- Here are most of them, highest-precedence to lowest.

Operator	Description
++ --	Postincrement, postdecrement
++ -- + - ! <i>(type)</i> sizeof	Preincrement, predecrement Unary positive, negative Logical not Type cast Well, it's the sizeof operator
* / %	Multiply, divide, mod
+ -	Add, subtract
< <= > >=	Relational operators

More C Operators

Operator	Description
== !=	Equals, not equals
&&	Logical and
	Logical or
? :	Ternary operator
= += -= *= ...	Assignment, and its friends
,	Comma operator

Meet Sizeof

- sizeof is an operator that tells you how much memory it takes to store something.
- It works for types:

```
sizeof( int )
```

- Or variables:

```
sizeof a
```

The parentheses aren't required, but remember, sizeof has fairly high precedence.

- Or other values:

```
sizeof( x * y )
```

- We're going to use this operator **a lot**.

How Much Memory : size_t

- C has a type for talking about how much memory you need
- It's called **size_t**
- In practice, this type is aliased to one of the types you've already seen.

Get it? Size – type?

– Like **unsigned long** on the common platform.

- There's a conversion specification for this type:

```
printf( "%zd\n", sizeof( x ) );
```

For printing

```
size_t s;  
scanf( "%zd", &s );
```

For reading

Meet The Comma Operator

- You can chain together multiple expressions with a comma
 - Sub-expressions get evaluated in order, left-to-right
 - The whole expression evaluates to the value of the last sub-expression.

```
int a = 5, b = 10, c = 20;  
int d;
```

```
d = ++a, b--, ++c;  
d = ( ++a, b--, ++c );
```

Here, d gets 6
(because = has higher precedence
than comma)

Here, d gets 22.

Meet The Comma Operator

- Mostly useful for packing multiple expressions in a space made for just one.

- Like in a for loop:

```
for ( int i = 0; i < n; i++, x-- ) {  
    ...  
}
```

Java will let you do this

- Or in goofy situations where you want shorten your code.

```
if ( ! readInput() )  
    printf( "Bad input\n" ), exit( 1 );
```

... but not this.

Meet The Ternary Operator

- Java has this, but sometimes students don't get to see it until this class.
- It's an expression that works like an if/else statement.

test ? expr₁ : expr₂

```
// Choose the larger of a and b.  
c = a;  
if ( b > a )  
    c = b;  
  
// Choose the right word to print.  
if ( x == 1 )  
    printf("Price: %d dollar\n", x);  
else  
    printf("Price: %d dollars\n", x);
```

```
// Choose the larger of a and b.  
c = a > b ? a : b;  
  
// Choose the right word to print.  
printf("Price: %d %s\n", x,  
       x == 1 ? "dollar" : "dollars");
```


Type Conversion

- C can convert between its various basic types (int, float, unsigned short, long, bool, etc.)
 - This can be specified **explicitly**, via a **type cast**
 - Or **implicitly**, by combining types in an expression, or providing one type where the compiler expects another

```
unsigned int ui;  
char c;  
short s;  
double d;
```

```
...
```

```
c = (char) ( d * (double) ( ui + (unsigned int) s ) );  
c = d * ( ui + s );
```

We're explicitly specifying every individual conversion.

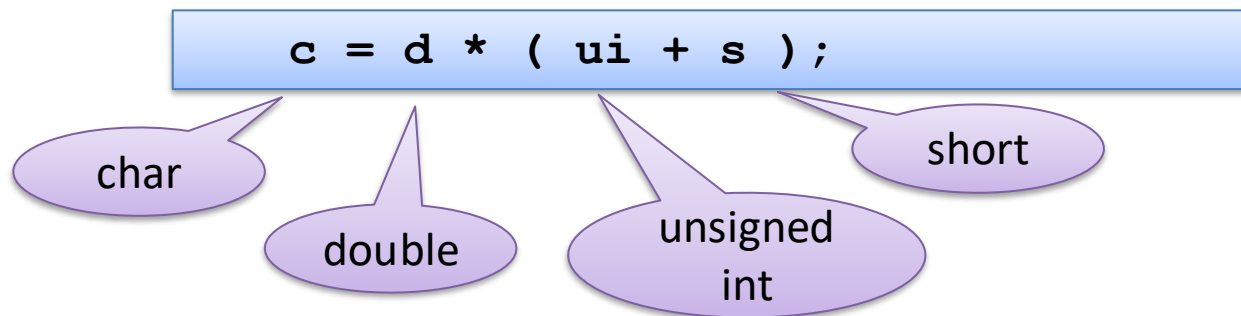
Here, the type conversions are implicit.

The Type Cast

- A type cast is a kind of operator
 - It lets the programmer specify a type conversion
- Syntax: `(destination_type) expression`
- Example: `c = (unsigned char) d;`
- Remember, the type cast has **very high precedence**
- A special case, casting to void
 - We can tell the compiler we don't need the result of an expression
 - Example: `(void) f( x );`
 - Why? Mostly to document behavior or suppress compiler warnings.

Implicit Conversions

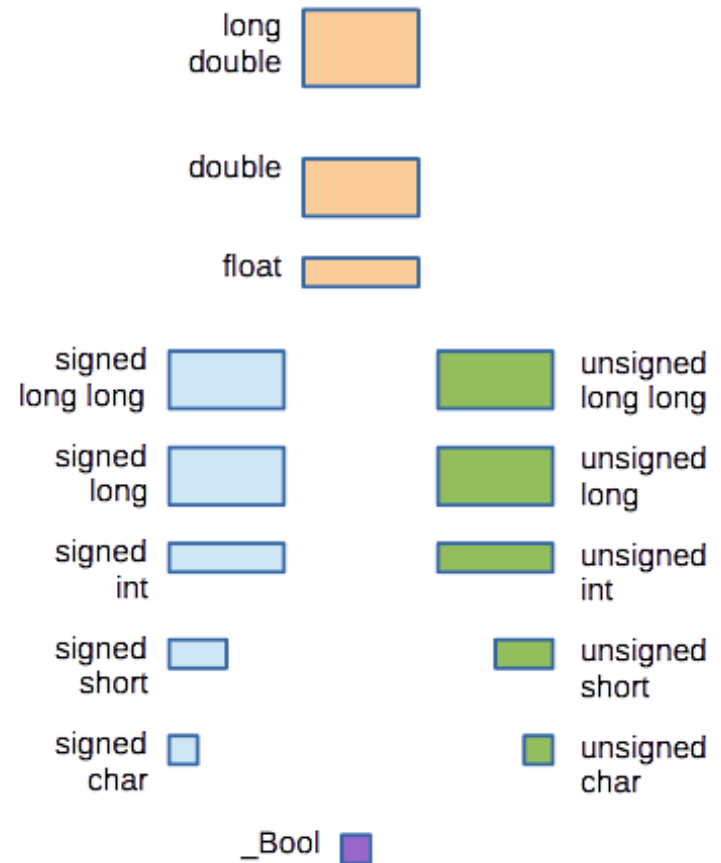
- Internally, the hardware typically can't perform arithmetic on mixed types
- C will automatically insert numeric conversions in order to evaluate an expression
- Lots more than Java would perform automatically.



- The standard defines a set of *Usual Arithmetic Conversions*

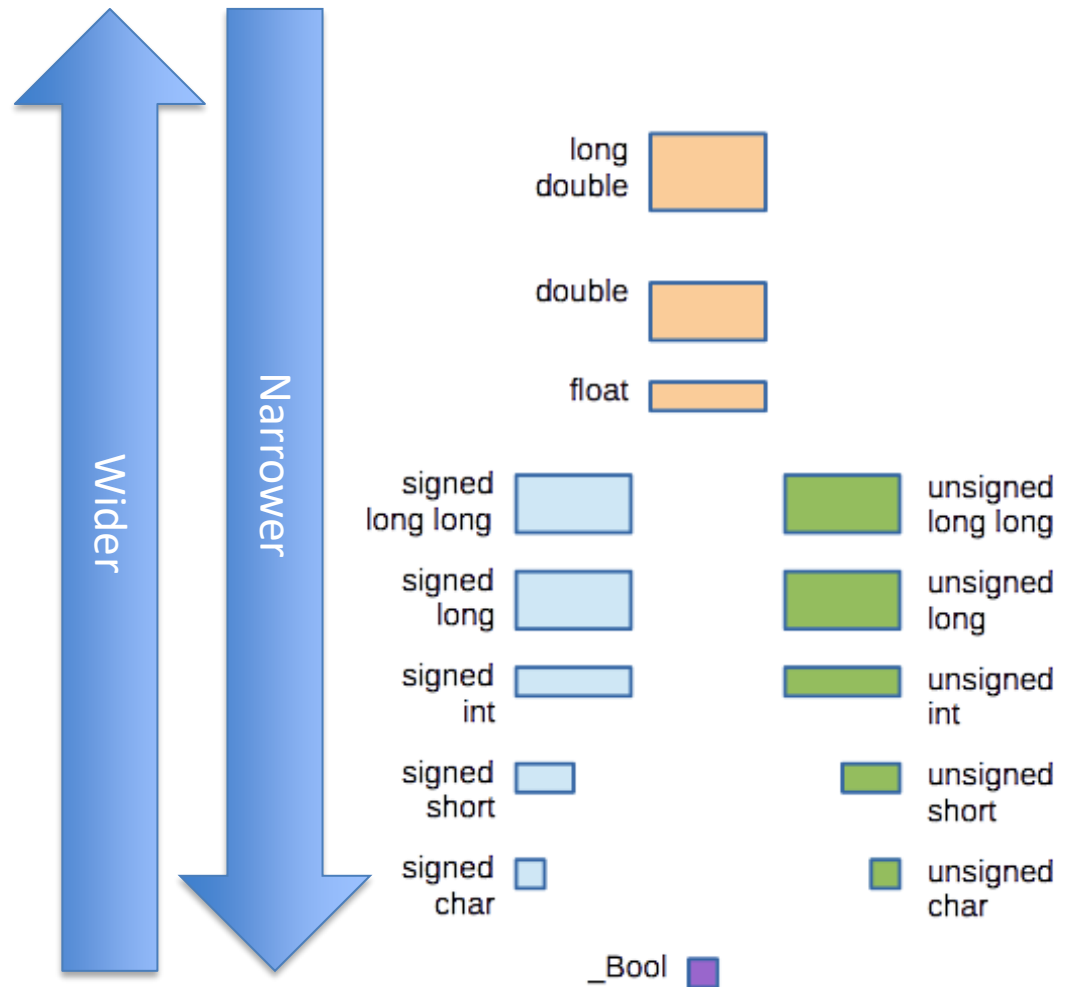
Usual Arithmetic Conversions

- The capacity of each type can depend on the compiler and the platform.
- C describes rules general rules that can be applied to any platform.
 - We will review these in general.
 - And, for the common platform.



Usual Arithmetic Conversions

- As we move up this chart, we tend to get greater capacity.
 - We can think of some types as *wider* or *narrower* than others.

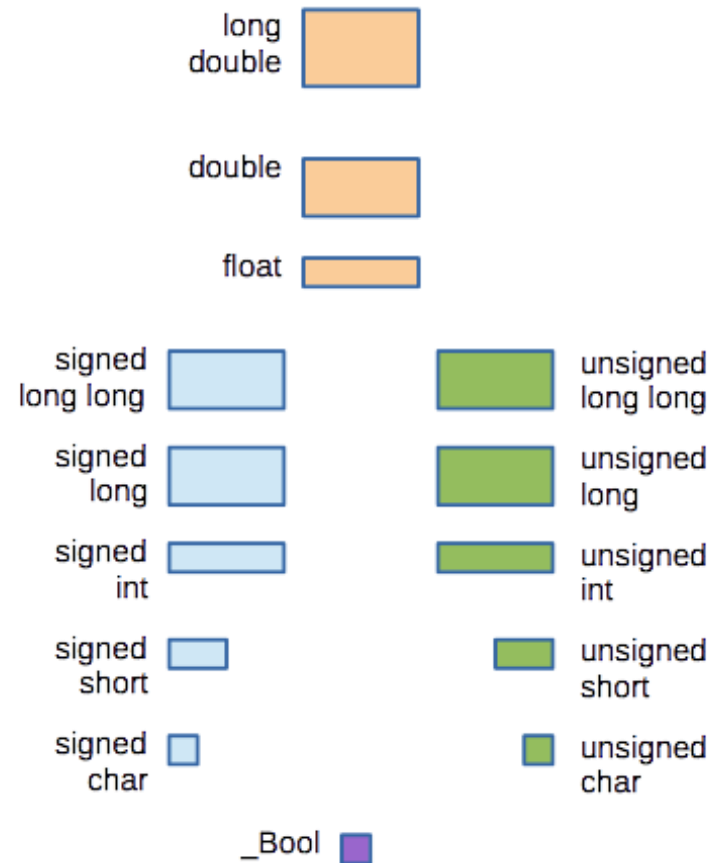


Usual Arithmetic Conversions

- Five rules to consider, depending on the two operand types.

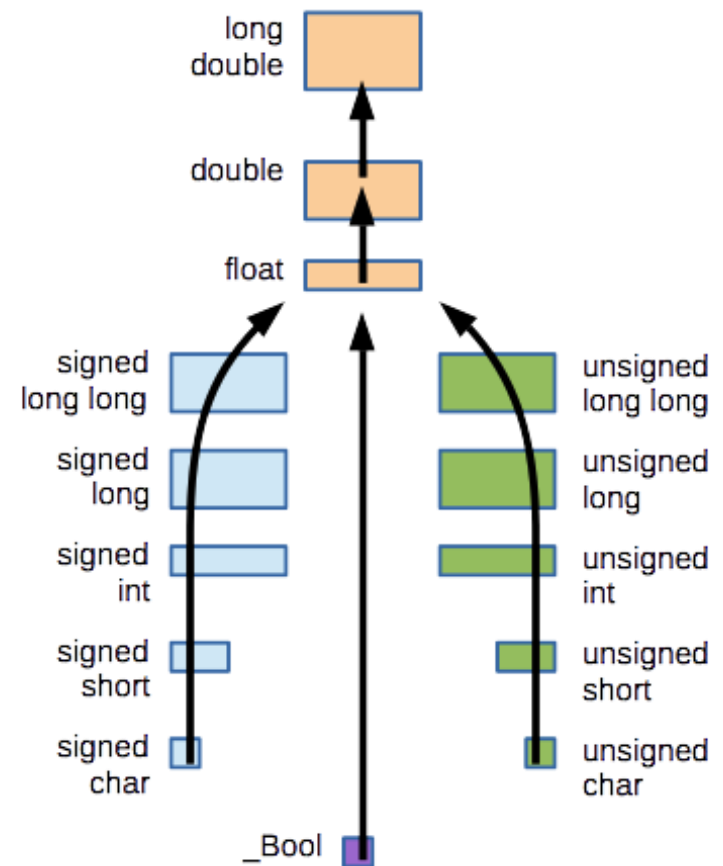
– For example:

```
int * float
unsigned char + char
short - unsigned long
bool / long long
short % long
```



Usual Arithmetic Conversions

- If an operand is a floating-point type, convert to the wider floating-point type.
 - If one operand is a floating-point type, convert to that.
 - If both are floating-point types, convert to the wider one.



Examples

`int * float`



`float * float`

`double + unsigned char`

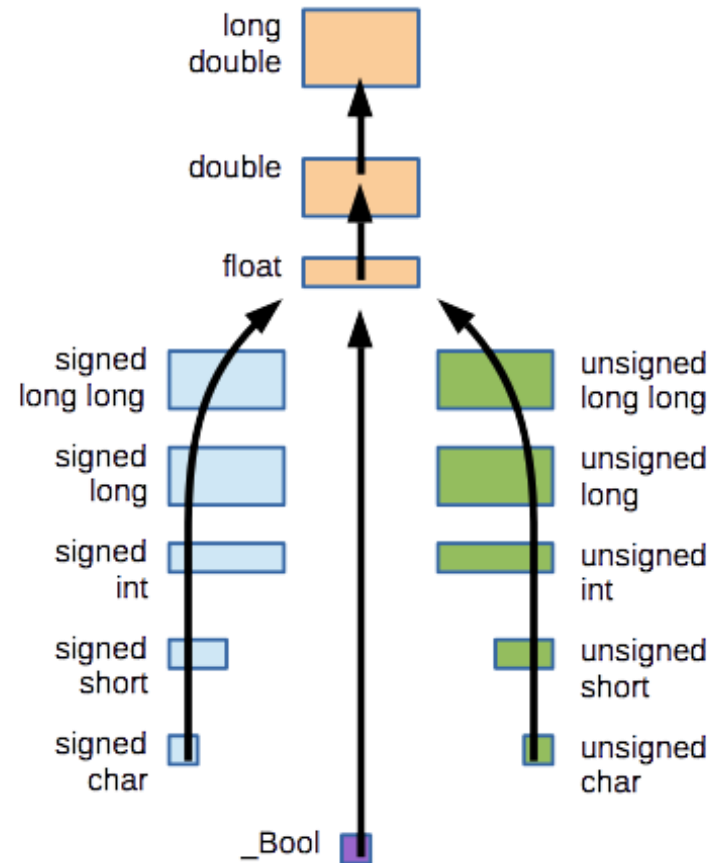


`double + double`

`float - long double`

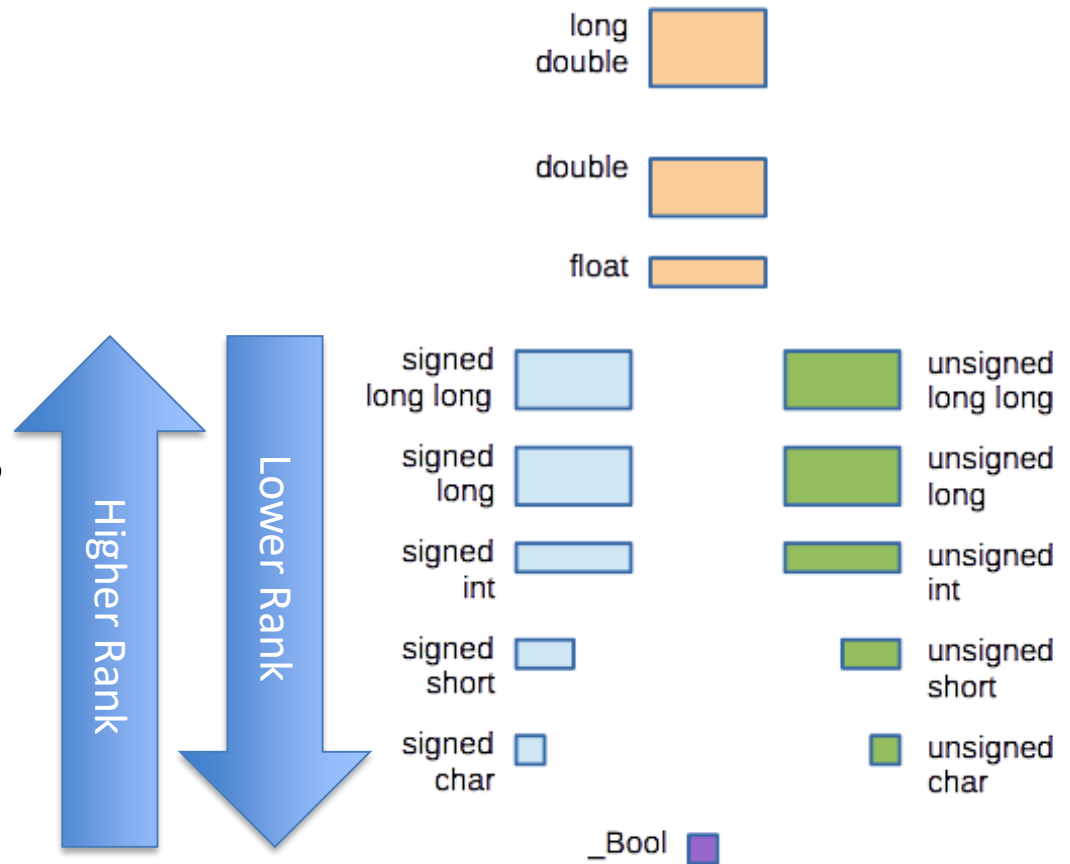


`long double - long double`



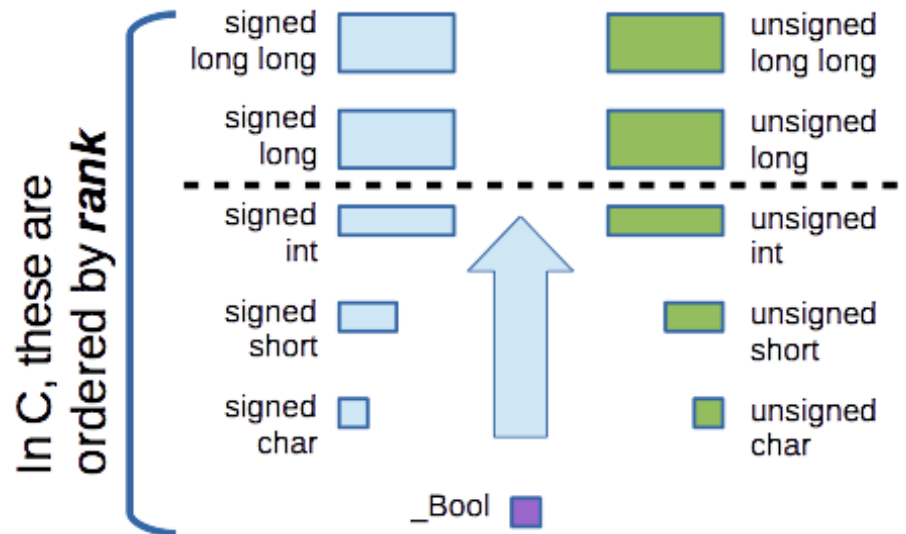
Usual Arithmetic Conversions

- OK. What if neither type is real-valued?
- C uses the *rank* to describe the relative size of its integer types.



Usual Arithmetic Conversions

- Otherwise, if either operand rank is lower than int, promote to (at least) the rank of int.

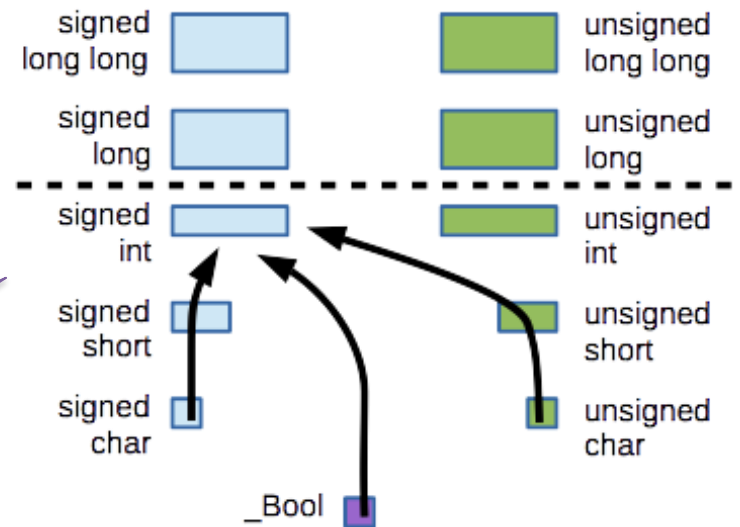


Usual Arithmetic Conversions

- Lower ranks are promoted to **signed int** if that is large enough to hold all values from lower ranked types.
 - This is how the common platform works.

The common platform is like this.

An int can hold any value stored as a short, unsigned short, signed char, unsigned char, or `_Bool`.



Examples

`char * unsigned short`



`int * int`

`bool + short`

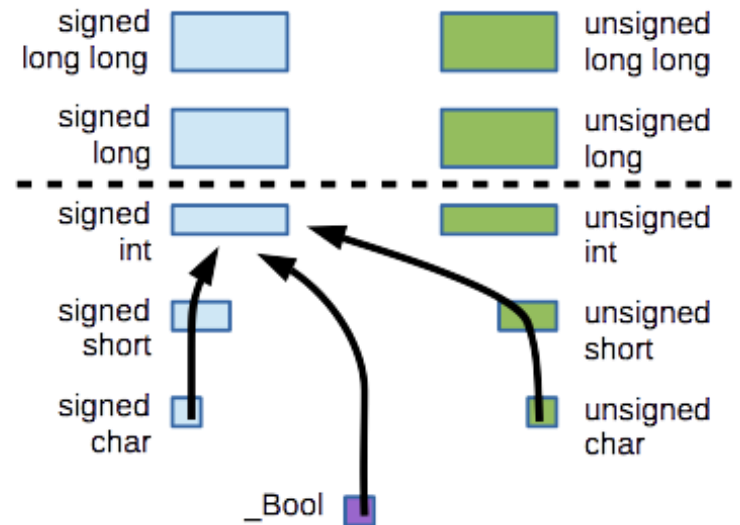


`int + int`

`char - unsigned char`

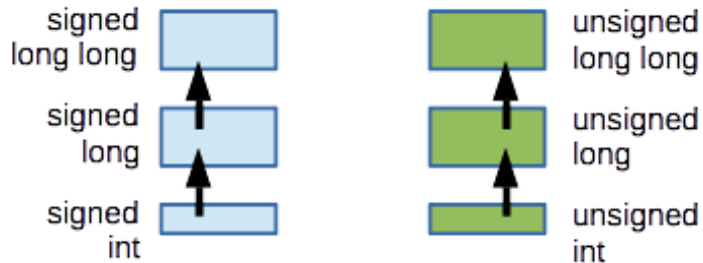


`int - int`



Usual Arithmetic Conversions

- If both types have the same signed-ness, promote to the higher-ranked type.

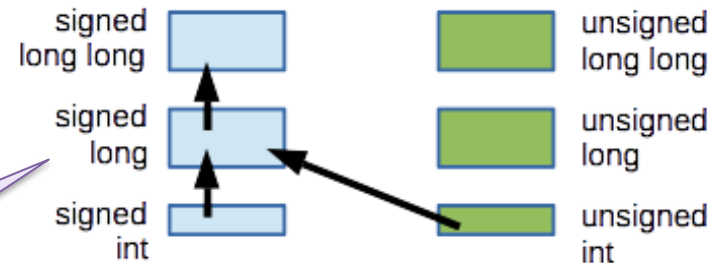


- What's left?
Just these six types with mixed signed-ness.

Usual Arithmetic Conversions

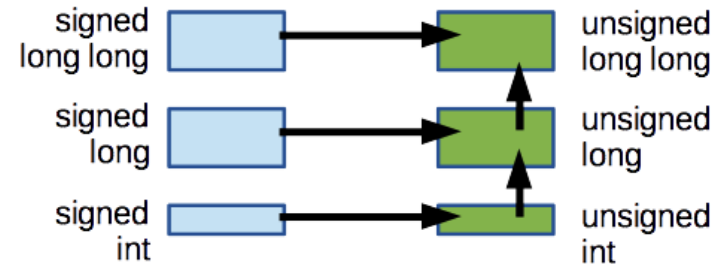
- Otherwise, if the signed type can store every value the unsigned type can, convert to the signed type.

The common platform is like this (long can hold anything an unsigned int can)



Usual Arithmetic Conversions

- Otherwise, if the signed type isn't large enough, use the **unsigned** version of the **higher-ranked** type.
- So, given two equal-ranked types of different signed-ness, C will use the **unsigned** type.



```
unsigned int x = 999;
while ( x ) {
    x = x - 1;
}
```

unsigned int

int

That makes sense.
You probably want
unsigned here.

Usual Arithmetic Conversions

- But, it's easy to make some mistakes.

```
unsigned int x = 999;  
if ( x > -1 ) {  
    printf( "You might think this will be printed\n" );  
}
```

This comparison will be done as **unsigned int**.

Usual Arithmetic Conversions

`long * unsigned int`



`long * long`

`int + unsigned long`

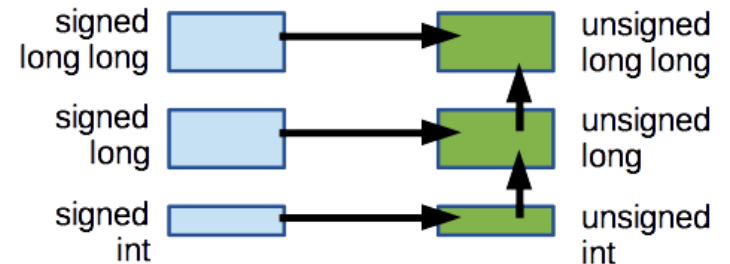
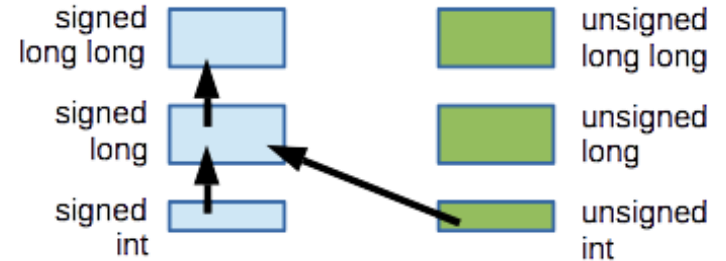


`unsigned long + unsigned long`

`long long - unsigned long`



`unsigned long long - unsigned long long`



Fun with Usual Arithmetic Conversions

- So, in the given code what is the type of the **red operator**? (what type will its operands be converted to so it can be evaluated)

```
signed char sc;  
long int l;  
short s;  
sc * s + l;
```

- How about here?

```
float f;  
int i;  
unsigned char uc;  
i + uc * f;
```

Fun with Usual Arithmetic Conversions

- So, in the given code what is the type of the **red operator**? (what type will its operands be converted to so it can be evaluated)

```
signed char sc;  
long int l;  
short s;  
sc * s + l;
```



If either operand
rank is lower than
int ...

- How about here?

```
float f;  
int i;  
unsigned char uc;  
i + uc * f;
```

Fun with Usual Arithmetic Conversions

- So, in the given code what is the type of the **red operator**? (what type will its operands be converted to so it can be evaluated)

```
signed char sc;  
long int l;  
short s;  
sc * s + l;
```

- How about here?

```
float f;  
int i;  
unsigned char uc;  
i + uc * f;
```

Don't forget about precedence here.

If an operand is a floating-point type

...

Fun with Usual Arithmetic Conversions

- How about here?

```
int i;  
unsigned int ui;  
long int l;  
i / ui * l;
```

- How about each operator from our original example?

```
float f;  
long long ll;  
int i;  
short s;  
s = f + ( ll * i );
```

Fun with Usual Arithmetic Conversions

- How about here?

```
int i;  
unsigned int ui;  
long int l;  
i / ui * l;
```

If the signed type
isn't large enough
...

If the signed type
can store every
value in the
unsigned ...

- How about each operator from our original example?

```
float f;  
long long ll;  
int i;  
short s;  
s = f + ( ll * i );
```

Fun with Usual Arithmetic Conversions

- How about here?

```
int i;  
unsigned int ui;  
long int l;  
i / ui * l;
```

- How about each operator from our original example?

```
float f;  
long long ll;  
int i;  
short s;  
s = f + ( ll * i );
```

If both types have
the same signed-
ness ...

If an operand is a
floating-point type
...

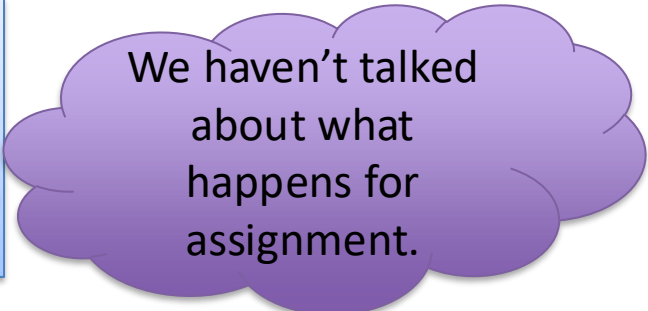
Fun with Usual Arithmetic Conversions

- How about here?

```
int i;  
unsigned int ui;  
long int l;  
i / ui * l;
```

- How about each operator from our original example?

```
float f;  
long long ll;  
int i;  
short s;  
s = f + ( ll * i );
```



We haven't talked about what happens for assignment.

Assignment and Implicit Conversion

- Implicit conversions can also happen in assignment
- Here, it's easy
 - the source value just gets converted to the destination type.

```
char c;  
int i;  
double d;  
d = i * c;
```

Resulting int
converted to
double for
assignment.

```
long l;  
float f;  
bool b;  
b = l - f;
```

Resulting float
converted to bool
for assignment.

Function Parameters and Implicit Conversion

- In C, function parameters are also easy.
 - Since there's **no function overloading**.
- Parameters values get converted to the parameter types expected by the function.
- Return value gets converted to the return type.

```
int f( double x, bool y )  
{  
    if ( y )  
        return x + 1.0;  
    return x - 1.0;  
}
```

Converted to an int

```
...  
short s = f( 'a', 3.14 );  
...
```

Converted to
double

Converted to bool